COMPSYS 701

Group Project 1

Jonathan Cilliers jcil831

Pulasthi Lenaduwa plen276

Wenjing Yuan wyua331

Department of Electrical, Computer, and Software Engineering

University of Auckland

Auckland, New Zealand

Date of Report

April 21, 2026

1 Project Design

Our project is composed of 3 layers. The top level defines the ReCOP and connections to the external interfaces and peripherals, this includes the clock signal from the Nios Board as well as inputs and outputs to the keys, switches, LEDs and seven-segment displays, as well as logic enabling the system to enter a debugging mode. It is also where connections to the NoC and other processors will be added during the development of GP2. The layer below the top level is the ReCOP itself, wherein we define connections between our core components, the control unit and datapath. The control unit is a single component, consisting of an FSM to manage the multicycle logic sequence that determines which control signals are necessary to complete the current instruction. Finally, the datapath contains the components used to store and operate on the data according to the signals given by the control unit. In our implementation, these components consist of the program and data memory, a register file, and the ALU.

2 Implementation Overview

2.1 Datapath

The datapath process can be seen as beginning with the updating of the instruction. On the rising clock edge, it can either continue or break the flow of the program. Breaking the flow of the program occurs either when it is reset (from a key input of the board), or the program counter (PC) is set to the operand as with a jump instruction. If the flow of the program is continuing normally instead then, depending on the control signal inputs, we will load 16 bits into either the top half or the bottom half (operand) of the instruction (for a combined 32-bit instruction) before incrementing the PC. The instruction register (IR) is also further broken down into the opcode, addressing mode, RZ index, and RX index for the top half, and the operand for the bottom half. From the top half of the IR, the addressing mode (2 bits) and opcode (6 bits) are separated to be sent as signals to the control unit.

The other function of the datapath is to direct the remaining control signals received from the control unit and input signals determined from the instruction and the outputs of other components to the appropriate component. For the control signals, all components receive the clock signal, whilst the register file and ALU also receive the reset signal. In addition to this, four signals are sent to the ALU, one 3-bit signal to determine the operation (add, sub, and, or), two bits to select the first operand, one bit to select the second operand, and a final single bit to indicate if the z flag should be cleared.

Signal	Value	Action
alu_operation	000 (alu_add)	ALU will add operands
	001 (alu_sub)	ALU will subtract operand 1 from operand 2
	010 (alu_and)	ALU will AND operands
	011 (alu_or)	ALU will OR operands
	Others	Not implemented, result set to zero
alu_op1_sel	00	Operand 1 set to RX
	01	Operand 1 set to instruction operand
	10	Unused, operand 1 set to '1'
	Others	Operand 1 set to zero
alu_op2_sel	0	Operand 2 set to RX
	1	Operand 2 set to RZ
clr_z_flag	0	Z flag is not cleared
	1	Z flag is cleared (set to zero)

Four signals are sent to the register file, two 4-bit signals from the upper half of the instruction, selecting which registers to use as RZ and RX, one 3-bit signal to select which source the input data is coming from. This is either the operand, dprr, ALU, rz max, sip hold, er temp, or data memory, of these, currently the dprr and er temp are not implemented. The other signal sent to the register file is a 1-bit signal determining if data is to be written to RZ or not.

Signal	Value	Action
sel_z	XXXX	Converted to an integer value as an index to select RZ in the registers
sel_x	XXXX	Converted to an integer value as an index to select RX in the registers
rf_input_sel	000	Input is instruction operand
	001	Input is derived from dprr_res_reg
	011	Input is ALU result
	100	Input is rz_max
	101	Input is sip_hold
	110	Input is derived from er_temp
	111	Input is from data memory
	Others	Input is set to zero
ld_r	0	Data is not written
	1	Data input is written into RZ

Similarly, a 1-bit signal is sent to the data memory to enable writing.

dm_wren	0	Data is not written
	1	Data is written into address from input

For the remaining input signals, the ALU is sent one 16-bit signal from the datapath, containing the operand from the instruction, as well as the two 16-bit contents of RX and RZ from the register file. The register file receives four 16-bit signals, one containing the operand from the instruction, one from the ALU containing its result, one from the datapath containing the max between the values of RX and RZ, and one from the data memory. As well as unused signals for sip hold, and er temp. The data memory component is sent the lower 12 bits of the operand for when it is used as an address.

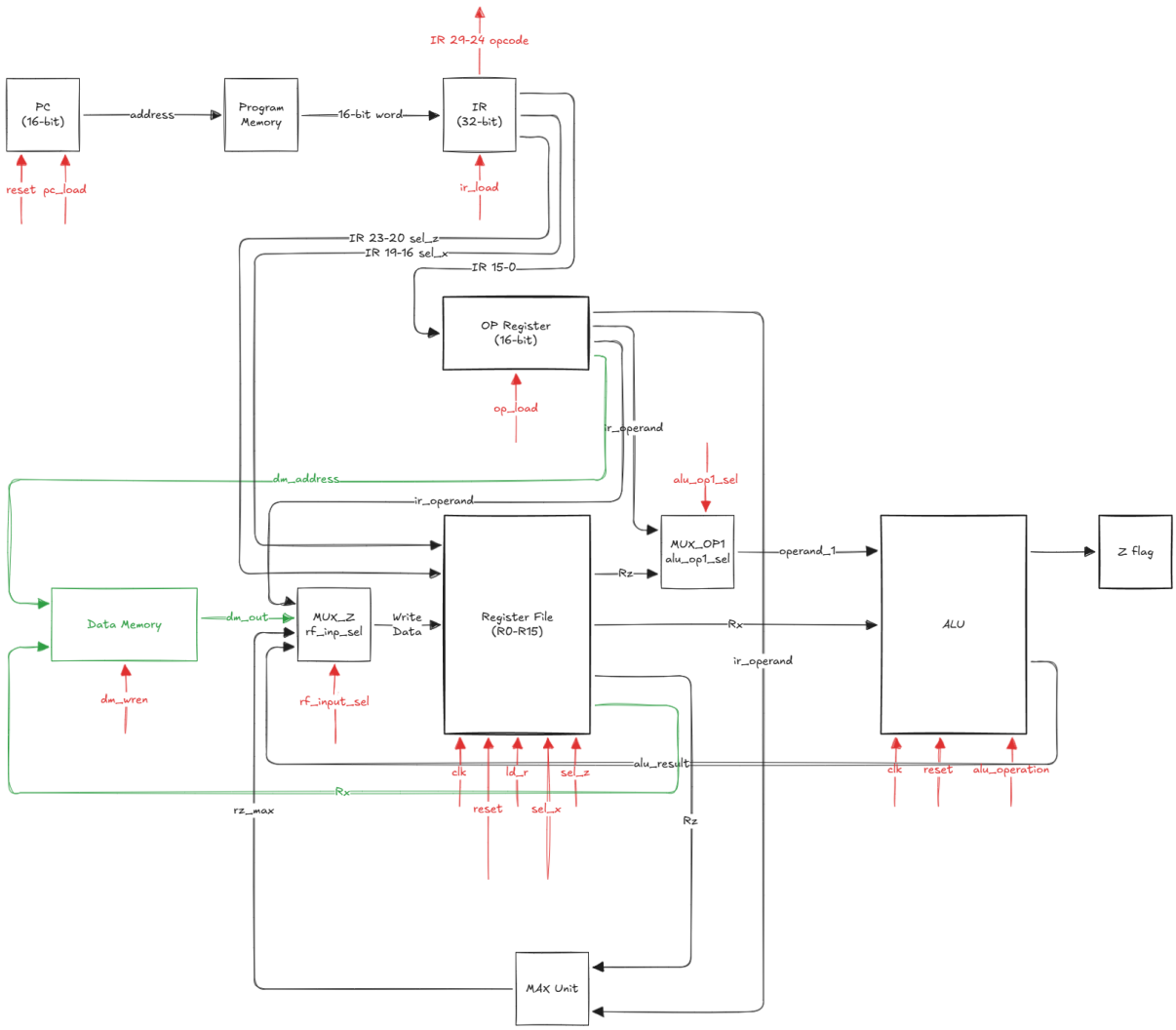

Figure 1: Datapath

2.2 Control Unit

The control unit maintains its state in the multicycle execution of an instruction using a Moore FSM. The FSM consists of six states, IDLE, FETCH_1, FETCH_2, WAIT_MEM, EXECUTE, and FETCH_JUMP. After entering the FETCH_1 state from IDLE, the FSM will complete an instruction in between 3 and 4 cycles. In state FETCH_1, the control signal ir_load is set to '1', resulting in the first 16 bits (31 downto 16) of the instruction being loaded into the IR from the program memory, these bits correspond to the addressing mode (2 bits), opcode (6 bits), rz - destination register (4 bits), and rx - source register (4 bits). These controls are interpreted as follows:

Addressing Mode (AM):

Value	Action
00 (Inherent)	No operand field used
01 (Immediate)	Operand is a literal value
10 (Direct)	Operand is a memory address
11 (Register)	Source is a register (RX)

Opcode:

000000	000010	011000	011100	001000
ldr	str	jmp	present	andr
001100	111000	000100	000011	010000
orr	addr	subr	subvr	clfz
111100	111110	111111	110100	010100
cer	ceot	seot	noop	sz
110110	111011	111010	110111	101000
ler	ssvop	ssop	lsip	datacall
101001	011110	011101	101010	Others
datacall2	max	strpc	sres	NULL/noop

Following this, FETCH_2 sets op_load to '1', loading the last 16 bits (15 downto 0) which contain the operand into the IR and OPR from the program memory. FETCH_2 will transition to WAIT_MEM if the instruction is loading data from a register or address, or storing data to DM, otherwise it will transition to EXECUTE. During WAIT_MEM, if data is being loaded from DM using an address stored in the registerfile the control signal dm_addr_sel is set to 10, the state will then move to EXECUTE. During the EXECUTE state, now that the necessary data is loaded, the control signals to perform the operation are sent to the datapath. If a jump is necessary the next state will be FETCH_JUMP, otherwise the FSM will return to FETCH_1.

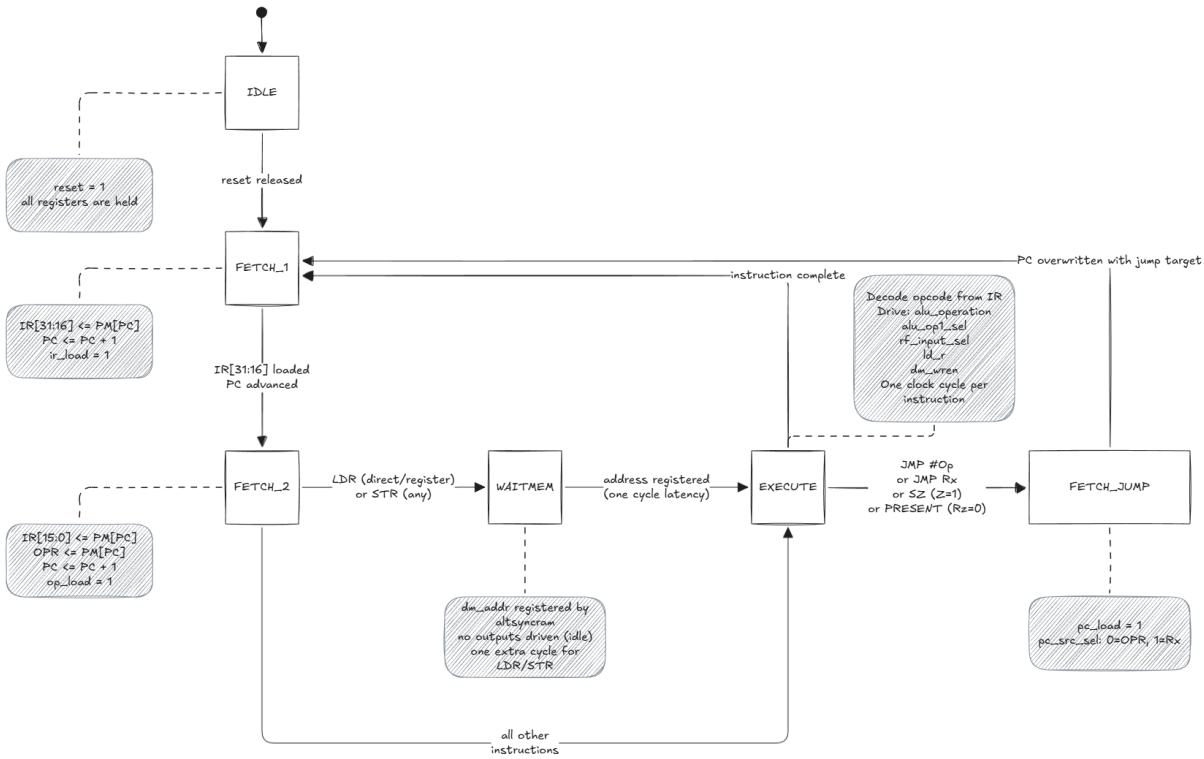

Figure 2: FSM

2.3 Assembler

To eliminate the dependency on MrASM and Cygwin, we developed a standalone Python assembler, asm.py. It compiles .asm source files into .mif (Memory Initialization Files) that can be directly loaded by Quartus. The instruction encoding strictly aligns with the field definitions in datapath.vhd: IR[31:30]=AM IR[29:24]=OP IR[23:20]=Rz IR[19:16]=Rx IR[15:0]=OPR Each 32-bit instruction is split into two consecutive 16-bit words for the MIF: the upper half-word is stored at the even address, and the lower half-word at the odd address.This mapping corresponds to the prog_mem configuration of WIDTH=16 and DEPTH=32768. The instruction set covers all GP1 mandatory items and the optional items already implemented by the CU. Addressing modes are hard-coded based on the current CU decoding capabilities. For example, SUBV is restricted to immediate values and SSOP to registers. Any forms unsupported by the CU will trigger an explicit error rather than passing silently.

2.4 Debugger

In the top level of our design a small amount of logic is added, allowing for the execution of the program on the ReCOP to be interrupted. This debug mode is enabled using switch 0 on the board. When the switch is high, the 1-bit signal debug_mode is sent to the control unit. Receiving this signal holds the processor at its current state, this gives us time to manually examine the values of the PC, AM, Opcode, z_flag, and RZ at the current cycle of the processor. Some information is visible using the LEDs. LED0 is on if debug mode is enabled, likewise LED1 shows the value of z_flag, LEDs 9 and 8 show AM, and LEDS 7 to 2 show the Opcode. The seven-segment displays are used to display the FSM's state as well as either the value of RZ or PC. The FSM's state is shown on HEX5, whilst RZ or PC are shown on HEX0-3 depending on whether switch 1 is high or low respectively.

3 Future Work and Extensions in GP2

For GP2, the datapath will be expanded upon to allow for communication between the ReCOP and additional processors on the chip via the NoC. To prepare for this, a few signals have been defined but not yet implemented related to the sop and dpcr, such as sop_reg, dpcr_reg, ssop_load, and dpcr_load among others. When complete, this will accommodate for registers which send and receive data to the NoC.

3.1 Acknowledgements

The design includes files supplied by the University of Auckland and Zoran Salcic.